

Universidade de Brasília
Departamento de Ciência da Computação
CIC0236 Teleinformática e Redes 2

RDT-UnB: relatório final

Evolução de SAW a protocolos de janela com RTT adaptativo

231013510 Atila Sousa Fernandes
231013645 Lucas Adeodato de Oliveira
222005395 Lurian Correia Lima

10 de novembro de 2026

Sumário

1 Introdução e evolução do projeto **1**
- 1.1 Entrega 1: Stop-and-Wait . . . 1
- 1.2 Entrega 2: janela deslizante . . . 1
- 1.3 Entrega 3: RTT adaptativo . . . 1
- 1.4 Linha do tempo das decisões . . . 1

2 RTT adaptativo — implementação **1**
- 2.1 Estimativa EWMA . . . 1
- 2.2 Justificativa dos parâmetros . . . 1

3 Comparação entre timeout fixo e adaptativo **2**
- 3.1 Interpretação . . . 2

4 Análise Wireshark **2**
- 4.1 Visão geral da sessão . . . 2
- 4.2 Eventos de retransmissão . . . 4
- 4.3 Correlação com o CSV . . . 4
- 4.4 Medição de RTT: Wireshark versus cliente . . . 5

5 Visão integrada — comparação final de todos os protocolos **6**

6 Conclusão **7**

1 Introdução e evolução do projeto

[Narrar a evolução técnica completa do projeto, da especificação inicial ao cliente final.]

1.1 Entrega 1: Stop-and-Wait

[Descrever o cliente SAW, o handshake, ACKs, checksum, timeout fixo e os resultados iniciais.]

1.2 Entrega 2: janela deslizante

[Descrever GBN e SR, janelas testadas, perdas simuladas e as dificuldades encontradas na implementação.]

1.3 Entrega 3: RTT adaptativo

[Explicar por que o timeout adaptativo foi necessário, quais problemas foram encontrados e como foram resolvidos.]

1.4 Linha do tempo das decisões

[Registrar decisões, experimentos, falhas e correções que mostrem a evolução do grupo, não apenas uma lista de funcionalidades.]

2 RTT adaptativo — implementação

[Descrever onde e como o estimador é atualizado no código e quais amostras são ignoradas após retransmissões.]

2.1 Estimativa EWMA

Apresentar a fórmula implementada:

$$\text{EstimatedRTT} \leftarrow (1-\alpha)\,\text{EstimatedRTT} + \alpha\,\text{SampleRTT}. \tag{1}$$

[Informar a fórmula completa do timeout, incluindo o desvio do RTT, e os valores escolhidos para α e β.]

2.2 Justificativa dos parâmetros

[Justificar α e β e mostrar, com dados do experimento, o impacto dos valores escolhidos.]

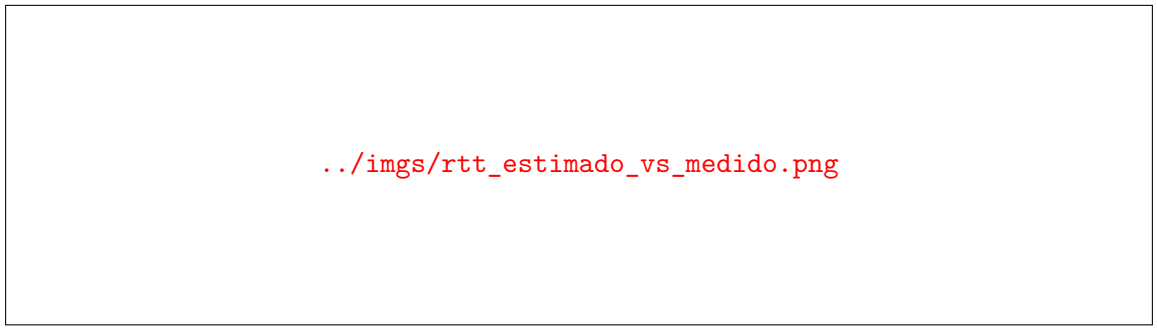


Figura 1: RTT estimado versus RTT medido por segmento durante a transferência.

[Inserir a análise da convergência do estimador, incluindo os trechos com variação, picos e possíveis perdas.]



Figura 2: Evolução do timeout adaptativo ao longo da transferência.

[Explicar como o timeout acompanha as variações de RTT e se houve estimativas excessivamente conservadoras ou agressivas.]

3 Comparação entre timeout fixo e adaptativo

[Escolher GBN ou SR e registrar a justificativa da escolha. Executar o mesmo protocolo, com a mesma janela, arquivo, semente e condições, usando timeout fixo e adaptativo sob perda de 5% e 10%.]

Tabela 1: Comparação entre timeout fixo e adaptativo.

Protocolo	Perda	Timeout	Retransmissões	Tempo total (s)	Throughput médio (kbps)
[GBN/SR]	5%	fixo	[n]	[s]	[kbps]
[GBN/SR]	5%	adaptativo	[n]	[s]	[kbps]
[GBN/SR]	10%	fixo	[n]	[s]	[kbps]
[GBN/SR]	10%	adaptativo	[n]	[s]	[kbps]

3.1 Interpretação

[Explicar em que situação o timeout adaptativo tem maior vantagem e em que situação o timeout fixo é suficiente. Relacionar os resultados às perdas, ao RTT e às retransmissões espúrias.]

4 Análise Wireshark

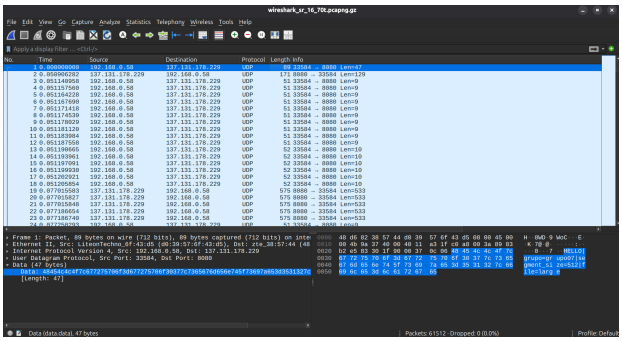
A captura foi feita durante a transferência do arquivo *large* (10 485 760 bytes, 20 480 segmentos de 512 bytes) com o protocolo Selective Repeat, janela de 16 segmentos e timeout fixo de 70 ms (`-protocol sr -window 16 -timeout 70`), no dia 09/11/2026. O arquivo `wireshark_sr_16_70t.pcapng.gz` tem 61 512 pacotes UDP e 36,148 s de duração, entre o cliente (`192.168.0.58`, porta 33584) e o servidor (`137.131.178.229`, porta 8080). O CSV da mesma execução é `metrics_grupo07_sr_20261109_213144.csv`.

4.1 Visão geral da sessão

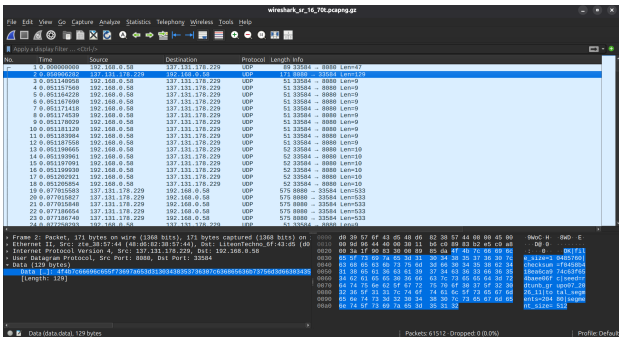
A Figura 3 reúne os cinco momentos da sessão, do `HELLO` ao `DONE`. O *Time* de cada pacote é relativo ao primeiro (`HELLO`).

Os intervalos principais são:

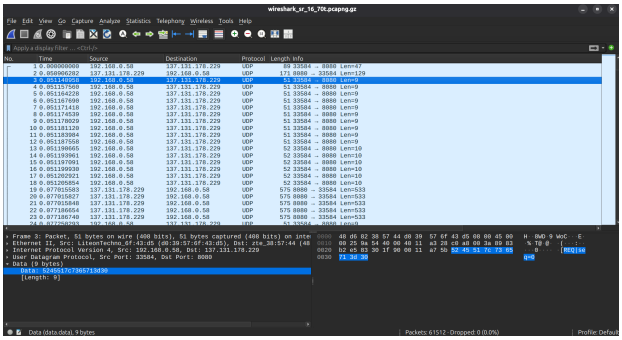
- `HELLO` → `OK`: 50,9 ms, maior que o RTT de dados (cerca de 26 ms).
- `OK` → primeiro `REQ`: 0,23 ms.
- Primeiro `REQ` → primeiro `DATA` (pacote 19): 25,87 ms, o RTT inicial da conexão.



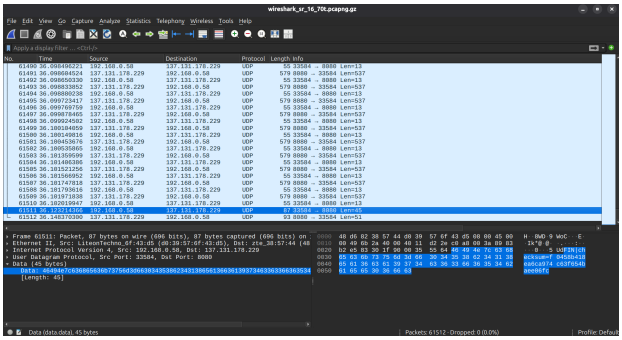
(a) `HELLO` (pacote 1, $t = 0$), 47 bytes de dados com grupo, tamanho de segmento e arquivo.



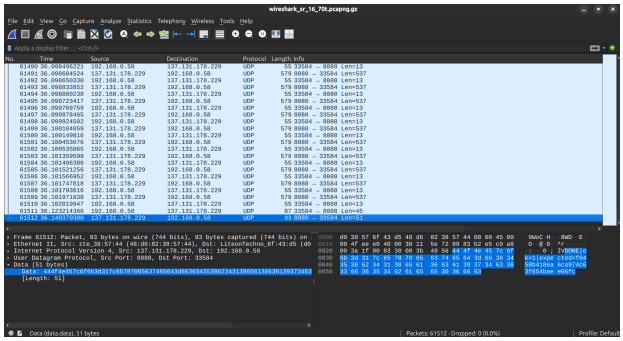
(b) `OK` (pacote 2, $t = 50{,}906$ ms) com `file_size`, `checksum`, `seed` e `total_segments=20480`.



(c) Primeiro `REQ|seq=0` (pacote 3, $t = 51{,}141$ ms). Os pacotes 3 a 18 são os 16 REQs da janela inicial, enviados em $\approx 65\,\mu$s.



(d) `FIN|checksum=...` (pacote 61 511, $t = 36{,}123$ s), enviado após o último segmento.



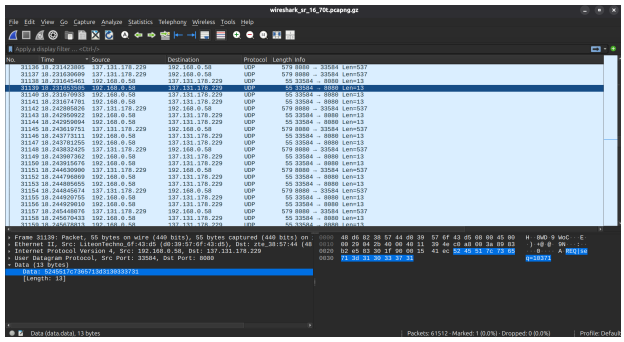
(e) `DONE|ok=1|expected=...` (pacote 61 512, $t = 36{,}148$ s), confirmando o MD5.

Figura 3: Visão geral da sessão completa: `HELLO` → `OK` → sequência de `REQ`/`DATA` → `FIN` → `DONE`.

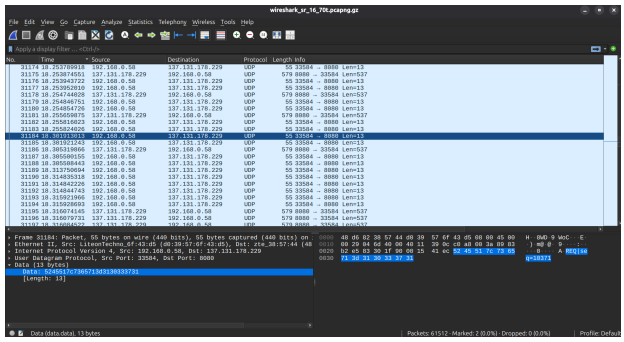
- Transferência: do pacote 3 ao 61 510, aproximadamente 36,05 s, igual ao tempo total que o cliente reporta.
- Último `REQ` → `FIN`: 21,2 ms; `FIN` → `DONE`: 25,2 ms, cerca de um RTT.

4.2 Eventos de retransmissão

O evento analisado é o do segmento 10 371. A Figura 4 mostra o `REQ|seq=10371` original e a retransmissão, ambos com 13 bytes de dados.



(a) REQ original: pacote 31 139, $t = 18{,}231653505$ s.



(b) REQ retransmitido: pacote 31 184, $t = 18{,}301913013$ s.

Figura 4: Retransmissão do segmento 10 371. Os dois pacotes estão marcados na captura.

Tabela 2: Evento de retransmissão do segmento 10 371 medido na captura.

Pacote	Frame	Tempo relativo (s)
REQ original	31 139	18,231653505
REQ retransmitido	31 184	18,301913013
Δt		0,070259508 (70,26 ms)

O intervalo de 70,26 ms corresponde ao timeout fixo de 70 ms configurado no cliente, com 0,26 ms de excesso. Entre os dois pacotes o servidor segue respondendo a outros segmentos da janela, como esperado no Selective Repeat: só o segmento 10 371 é reenviado. [Incluir um segundo evento de retransmissão (outro segmento, com REQ original, REQ retransmitido e Δt), pois o enunciado pede pelo menos dois.]

4.3 Correlação com o CSV

A Figura 5 mostra o trecho do CSV do cliente onde o campo `retransmissions` vale 1 e `timeout_occurred` vale 1. Nessas linhas o `rtt_ms` fica vazio: um segmento retransmitido não gera amostra de RTT, o que evita a ambiguidade de Karn. O `timeout_ms` permanece em 70 ms e o `rtt_estimated_ms` não muda (26,112 ms), pois não entrou amostra nova.

/home/lurian/Projects/RDT-UnB/results/entrega03/wireshark/metrics_grupo07_sr_20261109_213144.csv

Large file preview · Read-only

#	segment	timestamp	size_bytes	rtt_ms	rtt_estimated_ms	timeout_ms	throughput_kbps	retransmissions	timeout
2884	2883	2026-11-09T21:31:50.064631-03:…	512	25.644	26.192	70.000	159.723	0	0
2885	2884	2026-11-09T21:31:50.065231-03:…	512	25.552	26.112	70.000	160.303	0	0
2886	2885	2026-11-09T21:31:50.129942-03:…	512		26.112	70.000	45.885	1	1
2887	2886	2026-11-09T21:31:50.130942-03:…	512		26.112	70.000	45.388	1	1
2888	2887	2026-11-09T21:31:50.130974-03:…	512		26.112	70.000	45.380	1	1
2889	2888	2026-11-09T21:31:50.130988-03:…	512		26.112	70.000	45.670	1	1
2890	2889	2026-11-09T21:31:50.143844-03:…	512		26.112	70.000	39.990	1	1
2891	2890	2026-11-09T21:31:50.144842-03:…	512		26.112	70.000	44.502	1	1
2892	2891	2026-11-09T21:31:50.144873-03:…	512		26.112	70.000	44.501	1	1
2893	2892	2026-11-09T21:31:50.144887-03:…	512		26.112	70.000	44.501	1	1
2894	2893	2026-11-09T21:31:50.145743-03:…	512		26.112	70.000	44.097	1	1
2895	2894	2026-11-09T21:31:50.145776-03:…	512		26.112	70.000	47.179	1	1
2896	2895	2026-11-09T21:31:50.145792-03:…	512		26.112	70.000	47.639	1	1
2897	2896	2026-11-09T21:31:50.146441-03:…	512		26.112	70.000	48.075	1	1
2898	2897	2026-11-09T21:31:50.147058-03:…	512		26.112	70.000	48.161	1	1
2899	2898	2026-11-09T21:31:50.147198-03:…	512		26.112	70.000	48.584	1	1
2900	2899	2026-11-09T21:31:50.147416-03:…	512		26.112	70.000	49.492	1	1
2901	2900	2026-11-09T21:31:50.147645-03:…	512		26.112	70.000	49.716	1	1
2902	2901	2026-11-09T21:31:50.155402-03:…	512	25.445	26.028	70.000	160.977	0	0
2903	2902	2026-11-09T21:31:50.164891-03:…	512	33.927	27.016	70.000	120.730	0	0
2904	2903	2026-11-09T21:31:50.165687-03:…	512	34.704	27.977	70.000	118.028	0	0
2905	2904	2026-11-09T21:31:50.165845-03:…	512	34.849	28.836	70.000	117.537	0	0

20,480 rows 12 columns Delimiter Auto (Comma)

Figura 5: Linhas do CSV com segmentos retransmitidos: `rtt_ms` vazio, `retransmissions=1` e `timeout_occurred=1`.

A figura mostra a rajada dos segmentos 2885 a 2900 (16 segmentos, o tamanho da janela), por volta de 21:31:50. O evento do segmento 10 371 da Figura 4 está no mesmo arquivo:

```
10371,2026-11-09T21:32:03.241968-03:00,512,„26.046,70.000,55.478,1,1,16,sr,0
```

Para alinhar os relógios, o segmento 0 serve de referência: o primeiro `DATA` chega a $t = 77{,}016$ ms na captura e o CSV o registra em 21:31:45.013693, logo $t = 0$ da captura equivale a $\approx$21:31:44.9367. Com esse deslocamento, a retransmissão do pacote 31 184 ($t = 18{,}3019$ s) cai em $\approx$21:32:03.2386, a 3,4 ms do registro do CSV (21:32:03.241968). A diferença está dentro da imprecisão desse alinhamento (o instante exato do registro no cliente não é o do `sendto`). [Substituir a Figura 5 por um print da linha do segmento 10 371 (linha 10 372 do arquivo, entre as retransmissões 10 371 a 10 378), para que a imagem corresponda ao evento da Figura 4.]

4.4 Medição de RTT: Wireshark versus cliente

O RTT do segmento 0 foi medido na captura entre o `REQ|seq=0` (pacote 3, $t = 0{,}051140958$ s) e o primeiro `DATA` (pacote 19, $t = 0{,}077015583$ s):

$$\mathrm{RTT}_{\mathrm{Wireshark}} = 0{,}077015583 - 0{,}051140958 = 25{,}874625 \text{ ms}.$$



Figura 6: Saída do cliente ao final da transferência (resumo da sessão).

Tabela 3: RTT do segmento 0 no Wireshark e no CSV do cliente.

Segmento	RTT Wireshark (ms)	RTT CSV (ms)	Diferença (ms)
0	25,875	26,058	0,183

O CSV do cliente registra 26,058 ms para o segmento 0, 0,183 ms (0,7%) acima do valor da captura. A diferença tem sinal e tamanho esperados:

- O Wireshark marca o instante em que o pacote passa pela interface; o cliente marca o tempo em espaço de usuário, antes do `sendto` e depois do retorno do `recvfrom`. O cliente inclui a pilha do sistema e o escalonamento do processo nos dois sentidos, e por isso mede mais.

A medição do Wireshark é a que mais se aproxima do RTT de rede. Pelo resumo da Figura 6, o RTT médio no CSV é 28,1 ms (mín. 24,1 ms), e o estimado final é 27,0 ms. O valor de 25,9 ms do segmento 0 está na faixa observada, sem sinal de que o cliente distorça a medição.

5 Visão integrada — comparação final de todos os protocolos

[Selecionar o melhor cenário de cada protocolo e explicar o critério de seleção. Usar o arquivo large e condições comparáveis.]

Tabela 4: Resultados consolidados das três entregas.

Protocolo	Janela	Timeout	Throughput médio (kbps)	Retransmissões	Tempo total (s)
SAW	1	fixo	[kbps]	[n]	[s]
GBN	16	fixo	[kbps]	[n]	[s]
GBN	16	adaptativo	[kbps]	[n]	[s]
SR	16	fixo	[kbps]	[n]	[s]
SR	16	adaptativo	[kbps]	[n]	[s]

../imgs/comparacao_throughput.png

Figura 7: Throughput médio dos protocolos e configurações comparados.

[Interpretar o gráfico de barras como uma narrativa única da evolução do projeto, destacando ganhos, custos e o efeito do RTT adaptativo.]

6 Conclusão

[Responder objetivamente qual protocolo teve melhor desempenho no ambiente do projeto, considerando a rede real e RTT aproximado de 22 ms.]

[Avaliar, com os dados da Tabela 4, se o RTT adaptativo valeu a complexidade adicional de implementação.]

[Descrever o que o grupo implementaria de forma diferente se recomeçasse o projeto, incluindo melhorias de desenho, testes, instrumentação ou análise.]

Referências bibliográficas

[1] J. F. Kurose e K. W. Ross, *Computer Networking: A Top-Down Approach*, 8th ed. Hoboken: Pearson, 2021.

[2] J. Postel, "User Datagram Protocol," RFC 768, IETF, 1980. Disponível em: https://www.rfc-editor.org/rfc/rfc768.

[3] R. Rivest, "The MD5 Message-Digest Algorithm," RFC 1321, IETF, 1992. Disponível em: https://www.rfc-editor.org/rfc/rfc1321.